SIMATS ENGINEERING

ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO4**	**Assessment:**	**ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK**
Weightage:	**30%**	**Total Marks:**	**25**
CO4 Statement:	*Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.*		
Student Name:	Athisaya U	**Reg.No.:**	192571001
Department:	B.Tech CSE and Biosciences	**Date:**	21/08/2026

ANNEXURE A
INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization
   A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

2. Cloud Server Memory Bottleneck
   A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

3. Autonomous Vehicle Data Processing
   An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

4. AI Inference Edge Device
   An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

5. High-Performance Gaming Console
   A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

QUESTION 1

1. Smartphone Performance Optimization
A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

ANSWER

AIM

To analyze why application switching stalls occur and redesign the mobile memory hierarchy so that frequently reused application data is served by low-latency cache or DRAM instead of being repeatedly fetched from slower storage.

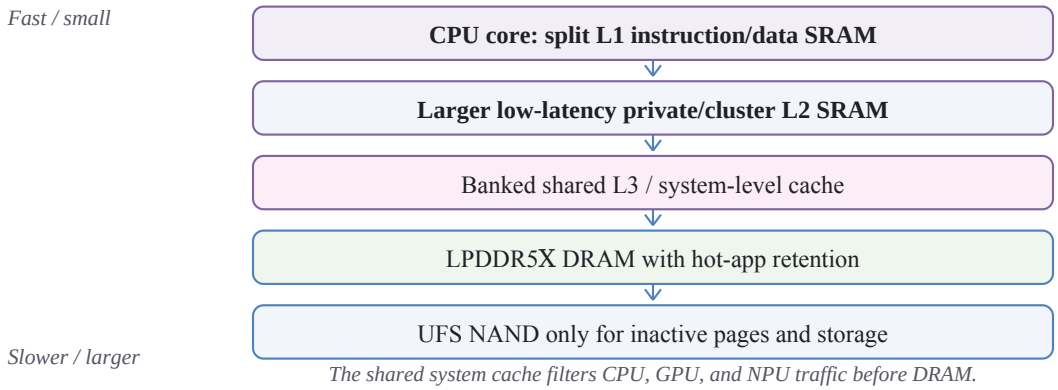
REQUIREMENTS

A four-level cache/DRAM model, conditional hit rates, access latency in nanoseconds, and Python 3 for the AMAT calculation. The numerical inputs are controlled design assumptions used to compare the existing and proposed hierarchies.

ROLES OF THE MEMORY LEVELS

Level	Technology and scope	Role during app switching	Design implication
L1	Per-core SRAM; smallest and fastest	Immediate instructions, stack data, and UI loops	Keep latency minimal; improve locality rather than only enlarging it
L2	Private or cluster SRAM	Captures displaced L1 working-set blocks	Moderate enlargement and better prefetching reduce L1 miss penalties
L3 / SLC	Shared on-chip SRAM	Retains code, graphics, and libraries reused across CPU/GPU/NPU	Increase capacity, banking, and quality-of-service allocation
DRAM	High-capacity LPDDR5X	Stores active and background app working sets	Use faster LPDDR5X, compression, and avoid unnecessary eviction to NAND

PROPOSED HIERARCHY



The shared system cache filters CPU, GPU, and NPU traffic before DRAM.

The redesign combines a higher shared-cache hit rate with faster LPDDR5X and an operating-system policy that keeps recently used application pages resident. The AMAT latencies and hit rates below are controlled inputs used to show the architectural improvement.

PYTHON CODE

```
def calculate_amat(levels):
    total = levels[-1]["latency"]
    for level in reversed(levels[:-1]):
        miss_rate = 1.0 - level["hit_rate"]
        total = level["latency"] + miss_rate * total
    return total

existing = [
    {"name": "L1", "latency": 1.0, "hit_rate": .88},
    {"name": "L2", "latency": 4.0, "hit_rate": .70},
    {"name": "L3", "latency": 12.0, "hit_rate": .55},
    {"name": "DRAM", "latency": 85.0, "hit_rate": 1.0}]
proposed = [
    {"name": "L1", "latency": 1.0, "hit_rate": .91},
    {"name": "L2", "latency": 4.0, "hit_rate": .78},
    {"name": "L3", "latency": 10.0, "hit_rate": .75},
    {"name": "DRAM", "latency": 65.0, "hit_rate": 1.0}]

before, after = calculate_amat(existing), calculate_amat(proposed)
reduction = (before - after) / before * 100
print(f"Baseline AMAT       : {before:.3f} ns")
print(f"Proposed AMAT       : {after:.3f} ns")
print(f"Latency reduction   : {reduction:.2f}%")
print(f"Access-rate proxy    : {1000/before:.1f} -> {1000/after:.1f} Maccess/s")
```

EXECUTION OUTPUT

Executed Python 3 | Q1 standalone simulation

```
Baseline AMAT       : 3.289 ns
Proposed AMAT       : 1.880 ns
Latency reduction   : 42.85%
Access-rate proxy    : 304.0 -> 532.0 Maccess/s
```

Figure 1(a): Output obtained by executing the Python code above.

PERFORMANCE ANALYSIS

Metric	Existing design	Proposed design
Conditional hit rates (L1 / L2 / L3)	88% / 70% / 55%	91% / 78% / 75%
L3 / DRAM latency	12 ns / 85 ns	10 ns / 65 ns
Calculated AMAT	3.289 ns	1.880 ns
Access-rate proxy (1 / AMAT)	304.0 M/s	532.0 M/s

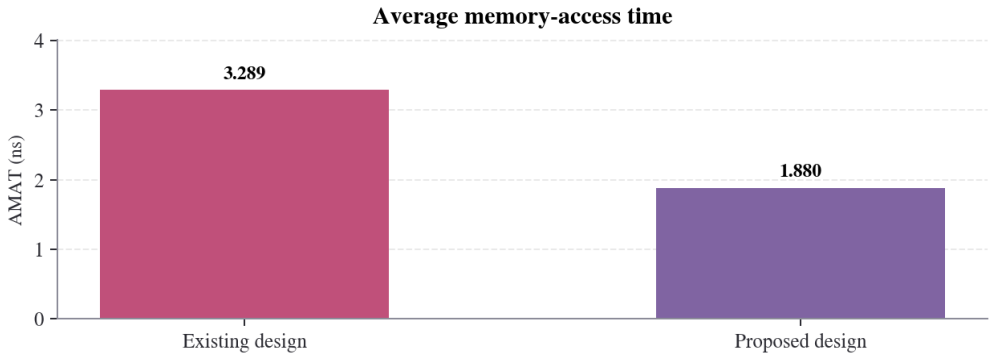


Figure 1(b): Controlled before-and-after AMAT comparison.

The model reduces AMAT by **42.85%.** The improvement comes mainly from raising the shared L3 hit rate, so fewer foreground switches reach DRAM. The inverse-AMAT value is a throughput proxy; real throughput also depends on memory-level parallelism and bus width.

RESULT: The redesigned L1-L2-L3-LPDDR5X hierarchy keeps hot app data closer to the processor, reduces the modeled average latency from 3.289 ns to 1.880 ns, and improves the access-rate proxy without relying on slow NAND during normal app switching.

QUESTION 2

2. Cloud Server Memory Bottleneck
A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

ANSWER

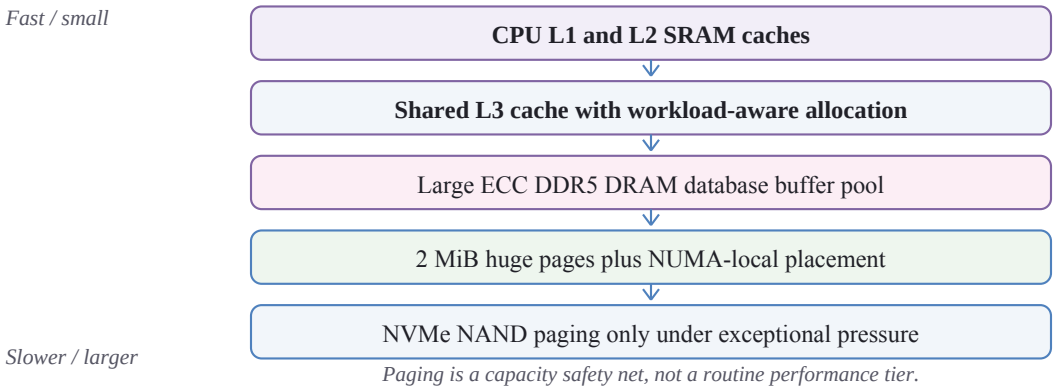
AIM

To distinguish cache misses from virtual-memory page faults in a database server and design a hierarchy that preserves a large in-memory working set while minimizing translation overhead and storage-backed paging.

CACHE HIERARCHY VERSUS VIRTUAL-MEMORY PAGING

Aspect	Cache hierarchy	Virtual memory and paging
Managed by	Hardware cache controller	Operating system and MMU
Granularity	Usually cache lines	Pages, commonly 4 KiB; huge pages can be 2 MiB or 1 GiB on x86
Normal role	Hide CPU-to-DRAM latency	Provide protection, translation, and capacity abstraction
Miss outcome	Fetch from a lower cache or DRAM	A major fault may fetch from NVMe/NAND and stall far longer
Best database use	Hot index/data reuse	Large mapped regions; avoid swap for latency-critical buffers

PROPOSED HIERARCHY



Paging is a capacity safety net, not a routine performance tier.

The database buffer pool should be sized so the hot indexes and pages remain in DRAM. Using 2 MiB huge pages for the stable buffer pool reduces TLB pressure, while NUMA-local allocation, asynchronous I/O, and cache partitioning limit interference between tenants.

ENGINEERING CONSTRAINTS

More DRAM increases cost and power, while a larger cache cannot hold the full database. Huge pages may waste capacity for small sparse mappings. Therefore, the practical design reserves huge pages for the stable buffer pool, retains ordinary pages elsewhere, and monitors LLC misses, major faults, swap activity, and P99 query latency.

PYTHON CODE

```
def calculate_amat(levels):
    total = levels[-1][0]
    for latency, hit_rate in reversed(levels[:-1]):
        total = latency + (1-hit_rate) * total
    return total

def effective_time(cache_ns, fault_rate, fault_penalty_ns):
    return cache_ns + fault_rate * fault_penalty_ns

existing_cache = [(1, .90), (4, .75), (14, .60), (90, 1.0)]
proposed_cache = [(1, .92), (4, .82), (12, .75), (75, 1.0)]
paging = {
    "existing": {"rate": .00020, "penalty": 200000},
    "proposed": {"rate": .00002, "penalty": 120000}}

cache_before = calculate_amat(existing_cache)
cache_after = calculate_amat(proposed_cache)
before = effective_time(cache_before, paging["existing"]["rate"],
                        paging["existing"]["penalty"])
after = effective_time(cache_after, paging["proposed"]["rate"],
                       paging["proposed"]["penalty"])

print(f"Cache-only AMAT      : {cache_before:.3f} -> {cache_after:.3f} ns")
print(f"Effective time       : {before:.3f} -> {after:.3f} ns")
print("Page-fault rate      : 0.020% -> 0.002%")
print(f"Effective reduction  : {(before-after)/before*100:.2f}%")
```

EXECUTION OUTPUT

Executed Python 3 | Q2 standalone simulation

```
Cache-only AMAT      : 2.650 -> 1.763 ns
Effective time       : 42.650 -> 4.163 ns
Page-fault rate      : 0.020% -> 0.002%
Effective reduction  : 90.24%
```

Figure 2(a): Output obtained by executing the Python code above.

PERFORMANCE ANALYSIS

Metric	Existing server	Proposed server
Cache-only AMAT	2.650 ns	1.763 ns
Major page-fault probability	0.020%	0.002%
Modeled page-fault penalty	200 us	120 us
Effective access time	42.650 ns	4.163 ns

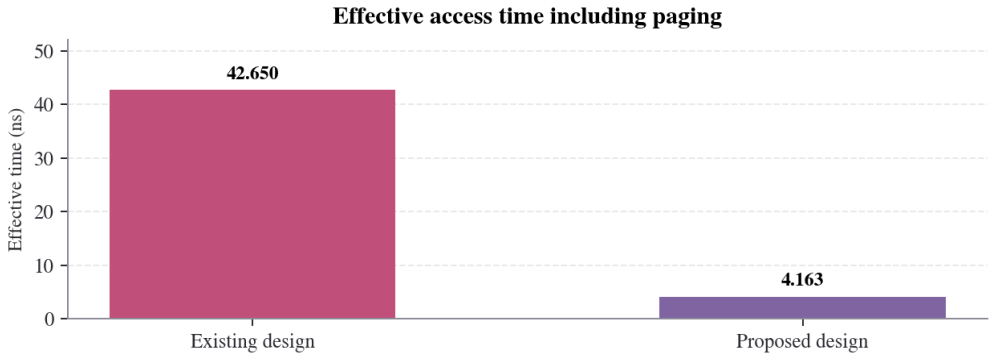


Figure 2(b): Page faults dominate the effective time even when they are rare.

The model reduces effective access time by **90.24%.** It shows why even rare storage-backed faults matter; cache misses and page faults must be monitored separately with P99 query latency.

RESULT: The recommended cloud hierarchy combines hardware caches, a sufficiently large DDR5 buffer pool, huge-page mappings, and NUMA locality. NVMe paging remains available for safety but is removed from the database's normal critical path, reducing the modeled effective time from 42.650 ns to 4.163 ns.

QUESTION 3

3. Autonomous Vehicle Data Processing
An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

ANSWER

AIM

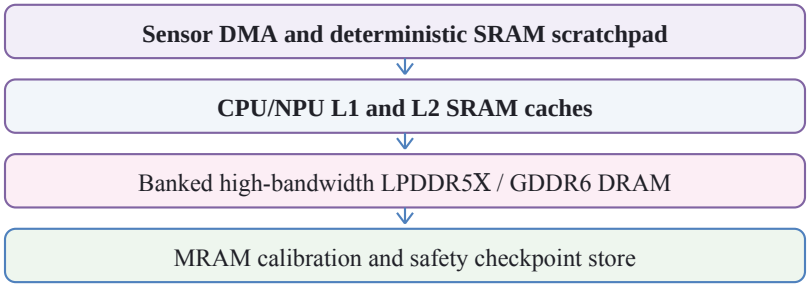
To evaluate SRAM, DRAM, and MRAM for a deterministic sensor-processing pipeline and design a hierarchy that supplies high bandwidth to perception accelerators while preserving safety-critical state across power interruptions.

TECHNOLOGY EVALUATION

Technology	Strengths	Limitations	Assigned role
SRAM	Lowest latency; no refresh; high bandwidth	Volatile, expensive, and low density	L1/L2 cache, scratchpad, DMA descriptors
DRAM	High capacity and streaming bandwidth	Volatile; refresh and row-access delay	Double-buffered camera, radar, and LiDAR frames
MRAM	Nonvolatile, durable, random access	Lower density/cost maturity; some row/write timings exceed DRAM	Calibration, maps, event logs, last-safe-state checkpoints

PROPOSED HIERARCHY

Fast / small



Slower / larger

Critical sensor frames stay in SRAM/DRAM; persistence traffic is isolated.

DMA places incoming sensor blocks into alternating DRAM buffers while the NPU processes the previous buffer. A reserved SRAM scratchpad holds the time-critical region, avoiding unpredictable eviction. MRAM stores compact calibration and safety state rather than the full live video stream because persistence is more important than minimum per-frame latency in this tier.

BANDWIDTH AND SAFETY CONTROLS

Memory-controller quality-of-service assigns guaranteed channels to perception and braking tasks, while infotainment receives lower priority. ECC protects DRAM and MRAM, bounded DMA queues prevent overwrite, and a watchdog verifies that the sensor-to-actuator deadline is met under peak traffic.

PYTHON CODE

```
def calculate_amat(levels):
    total = levels[-1]["latency"]
    for level in reversed(levels[:-1]):
        total = level["latency"] + (1-level["hit_rate"]) * total
    return total

existing = [
    {"memory": "SRAM L1", "latency": 1.0, "hit_rate": .85},
    {"memory": "SRAM L2", "latency": 4.0, "hit_rate": .65},
    {"memory": "DRAM", "latency": 90.0, "hit_rate": 1.0}]
proposed = [
    {"memory": "SRAM L1", "latency": 1.0, "hit_rate": .92},
    {"memory": "SRAM L2", "latency": 3.0, "hit_rate": .85},
    {"memory": "Banked DRAM", "latency": 60.0, "hit_rate": 1.0}]

before = calculate_amat(existing)
after = calculate_amat(proposed)
reduction = (before - after) / before * 100
access_before, access_after = 1000/before, 1000/after
print(f"Baseline AMAT       : {before:.3f} ns")
print(f"Proposed AMAT       : {after:.3f} ns")
print(f"Latency reduction   : {reduction:.2f}%")
print(f"Access-rate proxy    : {access_before:.1f} -> {access_after:.1f} Maccess/s")
```

EXECUTION OUTPUT

Executed Python 3 | Q3 standalone simulation

```
Baseline AMAT       : 6.325 ns
Proposed AMAT       : 1.960 ns
Latency reduction   : 69.01%
Access-rate proxy    : 158.1 -> 510.2 Maccess/s
```

Figure 3(a): Output obtained by executing the Python code above.

PERFORMANCE ANALYSIS

Metric	Existing path	Proposed path
L1 / L2 conditional hit rates	85% / 65%	92% / 85%
DRAM latency assumption	90 ns	60 ns
Calculated AMAT	6.325 ns	1.960 ns
Access-rate proxy	158.1 M/s	510.2 M/s

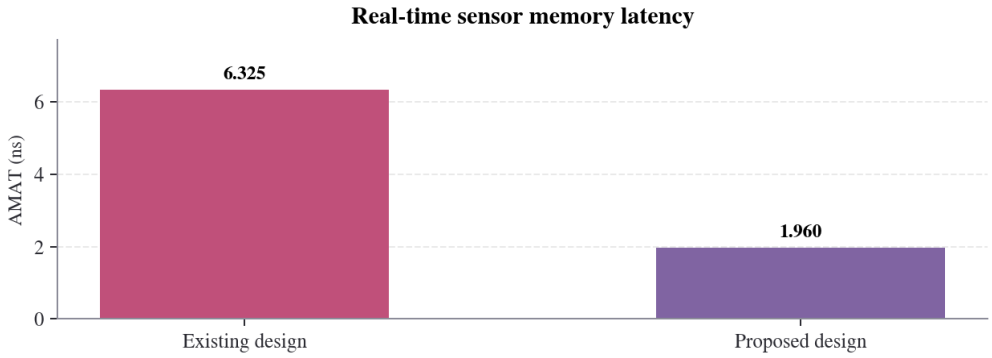


Figure 3(b): Modeled sensor-memory AMAT comparison.

The proposed SRAM locality and faster banked DRAM reduce modeled AMAT by **69.01%,** increasing the inverse-AMAT proxy by about 3.23x. MRAM is deliberately kept outside the per-frame critical path; its value is persistence and safe restart rather than replacing the fastest SRAM.

RESULT: A deterministic SRAM scratchpad and cache tier, double-buffered high-bandwidth DRAM, and a separate MRAM persistence tier provide both real-time throughput and retained safety state. The modeled critical-path AMAT falls from 6.325 ns to 1.960 ns.

QUESTION 4

4. AI Inference Edge Device
An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

ANSWER

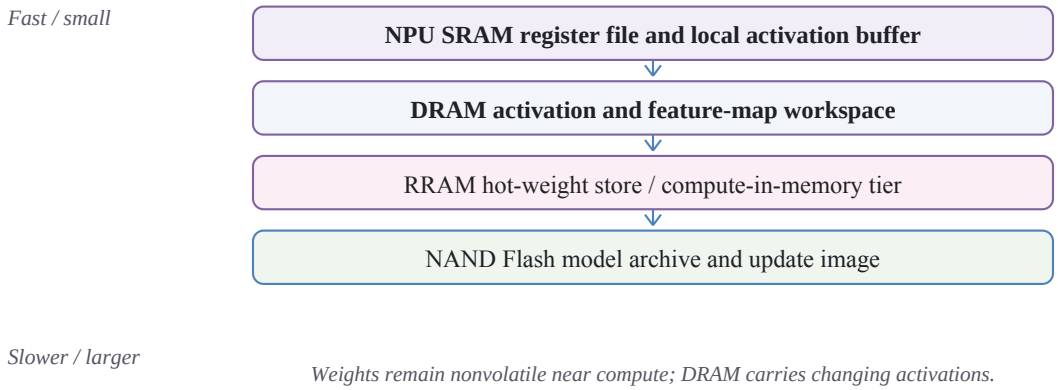
AIM

To reduce edge-AI model loading and repeated weight movement by assigning bulk storage, hot weights, and temporary activations to the memory technology best suited to each task.

TECHNOLOGY COMPARISON

Technology	Latency / power behavior	Capacity and endurance	Recommended role
NAND Flash	Slow block access; no refresh; efficient at rest	Highest density and low cost; finite program/erase endurance	Bulk model archive, firmware, datasets
DRAM	Fast random access and high bandwidth; refresh consumes power	Large working memory; volatile	Activations, feature maps, mutable runtime buffers
RRAM	Nonvolatile reads and possible compute-in-memory	Emerging density; write variability and endurance require management	Frequently used quantized weights and matrix-vector operations

PROPOSED HIERARCHY



At installation, complete models remain in NAND. Frequently used quantized layers are programmed into RRAM and retained across sleep, while DRAM is allocated to activations and layers that change during inference. This reduces boot traffic and repeated NAND-to-DRAM copies while keeping mutable data in a high-bandwidth workspace.

PRACTICAL LIMITATIONS

RRAM should be treated as an optimized hot tier rather than a universal replacement: programming is less frequent than reading, wear-leveling and calibration handle device variation, and an integrity-checked NAND copy remains the authoritative model. Power gating turns off inactive compute blocks while RRAM retains weights.

PYTHON CODE

```
def analyze_loading(model_mb, memory):
    load_ms = model_mb / memory["bandwidth_gbps"]
    moved_after_sleep = 0 if memory["nonvolatile"] else model_mb
    return {"load_ms": load_ms, "reload_mb": moved_after_sleep}

model_mb = 256
technologies = {
    "NAND_to_DRAM": {
        "bandwidth_gbps": 1.5,
        "nonvolatile": False,
        "role": "bulk model loading"},
    "RRAM_hot_tier": {
        "bandwidth_gbps": 8.0,
        "nonvolatile": True,
        "role": "retained hot weights"}}

nand = analyze_loading(model_mb, technologies["NAND_to_DRAM"])
rram = analyze_loading(model_mb, technologies["RRAM_hot_tier"])
reduction = (nand["load_ms"] - rram["load_ms"]) / nand["load_ms"] * 100

print(f"Model size           : {model_mb} MB")
print(f"NAND -> DRAM load    : {nand['load_ms']:.2f} ms at 1.5 GB/s")
print(f"RRAM hot-weight load : {rram['load_ms']:.2f} ms at 8.0 GB/s")
print(f"Load-time reduction  : {reduction:.2f}%")
```

EXECUTION OUTPUT

Executed Python 3 | Q4 standalone simulation

```
Model size           : 256 MB
NAND -> DRAM load    : 170.67 ms at 1.5 GB/s
RRAM hot-weight load : 32.00 ms at 8.0 GB/s
Load-time reduction  : 81.25%
```

Figure 4(a): Output obtained by executing the Python code above.

PERFORMANCE ANALYSIS

Metric	NAND-to-DRAM baseline	RRAM hot tier
Model size	256 MB	256 MB
Illustrative sustained read path	1.5 GB/s	8.0 GB/s
Calculated loading time	170.67 ms	32.00 ms
Weight persistence after power gating	No - reload required	Yes - weights retained

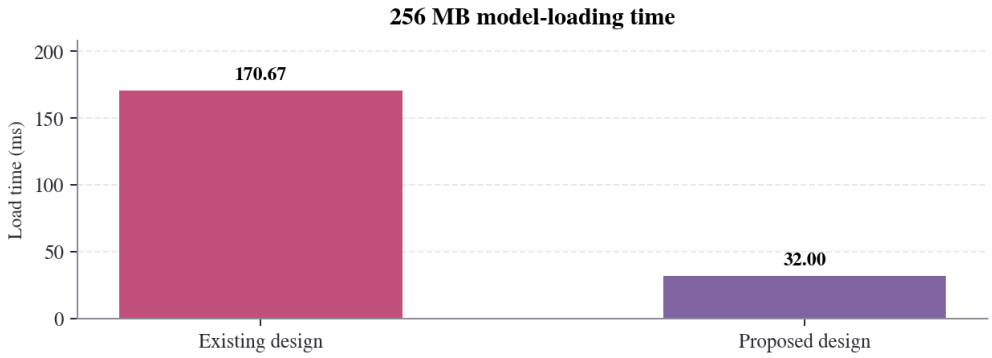


Figure 4(b): Illustrative model-loading time under equal model size.

The model reduces the 256 MB load from 170.67 ms to 32.00 ms, an **81.25%** reduction. These are architecture-level assumptions, not a product benchmark. The additional energy benefit comes from avoiding repeated movement of retained weights, while DRAM remains the correct tier for rapidly changing activations.

RESULT: The optimized NAND-RRAM-DRAM-SRAM hierarchy combines inexpensive bulk storage, nonvolatile hot weights near the NPU, and a fast volatile activation workspace. It cuts the modeled load time by 81.25% while reducing unnecessary weight transfers and preserving an update-safe NAND copy.

QUESTION 5

5. High-Performance Gaming Console
A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

ANSWER

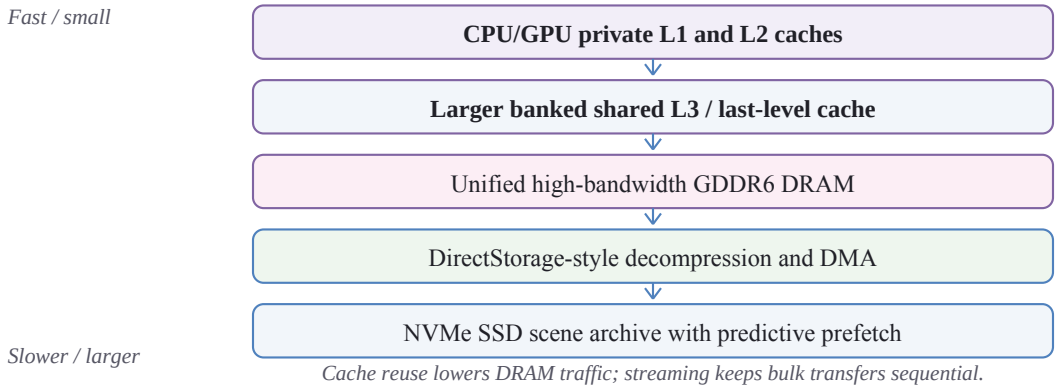
AIM

To determine whether frame drops originate from inadequate L3 locality or saturated graphics DRAM and propose a hierarchy that keeps reusable scene data on chip while sustaining large sequential asset transfers.

L3 CACHE VERSUS DRAM THROUGHPUT

Feature	L3 cache	Graphics DRAM
Technology	Shared on-chip SRAM	External GDDR6 DRAM
Latency	Much lower; optimized for cache-line reuse	Higher due to controller, row, and bus delay
Capacity	Tens of MB; cannot hold a complete large scene	Many GB; holds textures, geometry, and frame buffers
Throughput role	Eliminates repeated DRAM transactions	Supplies bulk streaming bandwidth
Main limitation	Capacity and contention	Bandwidth saturation and access locality

PROPOSED HIERARCHY



Cache reuse lowers DRAM traffic; streaming keeps bulk transfers sequential.

The design enlarges and banks L3, gives graphics and decompression queues explicit quality-of-service, and prefetches the next scene region into DRAM. Frequently sampled textures and geometry remain in L3, while one-use bulk assets bypass it when cache pollution would be worse. GDDR6 remains the high-capacity streaming tier and L3 remains the low-latency reuse tier.

FRAME-TIME VALIDATION METHOD

Record L3 hit rate, DRAM read bandwidth, decompression queue depth, and 1% low frame time. A redesign succeeds only if DRAM traffic remains below the sustainable budget during scene transitions and the frame-time spikes disappear; average frames per second alone can hide these stalls.

PYTHON CODE

```
def analyze_design(config, raw_demand):
    miss_rate = 1.0 - config["l3_hit_rate"]
    amat = config["l3_ns"] + miss_rate * config["dram_ns"]
    dram_traffic = raw_demand * miss_rate
    within_budget = dram_traffic <= config["dram_budget"]
    return amat, dram_traffic, within_budget

raw_demand = 800
existing = {
    "l3_ns": 15.0, "l3_hit_rate": .35,
    "dram_ns": 80.0, "dram_budget": 448}
proposed = {
    "l3_ns": 12.0, "l3_hit_rate": .65,
    "dram_ns": 65.0, "dram_budget": 448}

before, traffic_before, old_ok = analyze_design(existing, raw_demand)
after, traffic_after, new_ok = analyze_design(proposed, raw_demand)
reduction = (before - after) / before * 100

print(f"L3 + DRAM AMAT       : {before:.2f} -> {after:.2f} ns")
print(f"Raw scene demand     : {raw_demand} GB/s")
print(f"DRAM traffic         : {traffic_before:.0f} -> {traffic_after:.0f} GB/s (budget: 448 GB/
s)")
print(f"Latency reduction    : {reduction:.2f}%")
```

EXECUTION OUTPUT

Executed Python 3 | Q5 standalone simulation

```
L3 + DRAM AMAT       : 67.00 -> 34.75 ns
Raw scene demand     : 800 GB/s
DRAM traffic         : 520 -> 280 GB/s (budget: 448 GB/s)
Latency reduction    : 48.13%
```

Figure 5(a): Output obtained by executing the Python code above.

PERFORMANCE ANALYSIS

Metric	Existing console	Proposed console
L3 hit rate	35%	65%
L3 / DRAM latency	15 ns / 80 ns	12 ns / 65 ns
Calculated AMAT	67.00 ns	34.75 ns
DRAM traffic from 800 GB/s raw demand	520 GB/s	280 GB/s
Against 448 GB/s illustrative budget	Over budget	Within budget

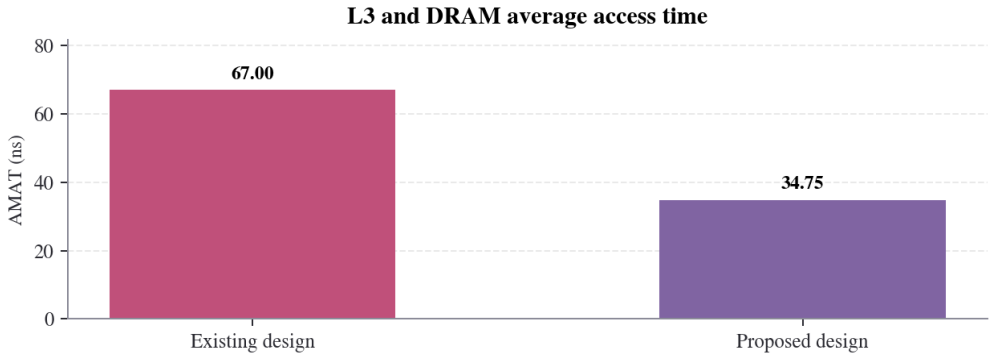


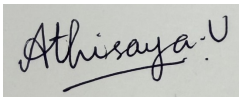
Figure 5(b): Modeled L3-plus-DRAM average access time.

The larger shared cache lowers AMAT by **48.13%.** At a 35% hit rate, the 800 GB/s request stream produces 520 GB/s of DRAM traffic and exceeds the 448 GB/s budget. Raising the hit rate to 65% cuts traffic to 280 GB/s.

RESULT: The enhanced L1-L2-L3-GDDR6-NVMe hierarchy addresses both causes of scene-loading frame drops: a larger banked L3 improves reuse and reduces AMAT from 67.00 ns to 34.75 ns, while predictive streaming and decompression keep the modeled DRAM traffic within its bandwidth budget.

***Code of Conduct**:*

I Athisaya U / 192571001 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis